PROJECT GUIDE

Inactivity Alarm

What it does, how it works, the technology behind it — and every concept explained in simple words.

Python 3 · Tkinter · Windows API · ctypes · winsound · pystray · PyInstaller · Inno Setup

A plain-English explanation of **what this project is, how it works, which technologies it uses and why**, plus the key concepts and keywords worth knowing.

VERSION
1.1.0

MADE BY
Ishant · © 2026 IMPS

UPDATED
November 2026

Contents

1. What the app does (in one minute) . . . 2
2. The big picture . . . 2
3. Technologies used, and why . . . 2
4. Files and what each one does . . . 3
5. How it works, step by step (the main pipeline) . . . 4
6. Every feature in simple words . . . 5
7. Key concepts you should know . . . 6
8. The build and release pipeline . . . 8
9. The testing pipeline . . . 9
10. Sharing the app: things to know . . . 10
11. Privacy by design . . . 10
12. Known limitations . . . 10
13. Glossary (quick reference) . . . 11
14. Common changes: where to edit . . . 12
15. Ownership, license and fingerprint . . . 12
16. Command cheat sheet . . . 13

01 What the app does (in one minute)

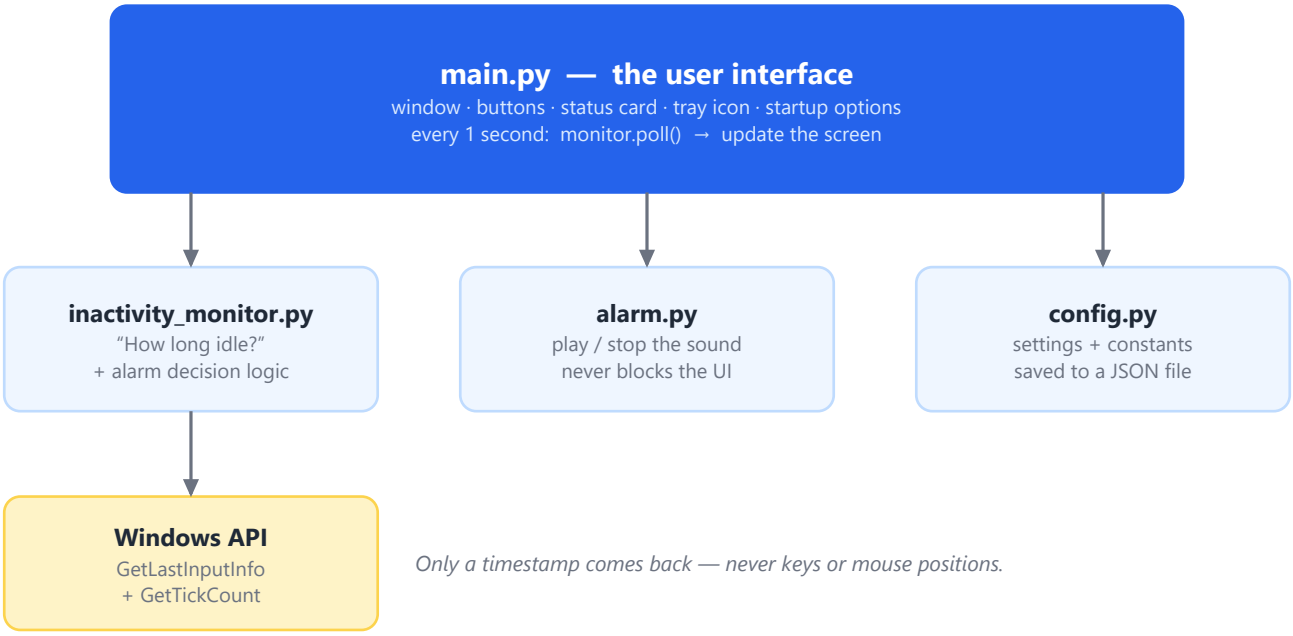
Inactivity Alarm watches whether you are using your computer. If nobody touches the **keyboard or mouse** for a set time (for example 5 minutes), it:

1. turns its window **red** and brings it to the front,
2. plays an **alarm sound**, and
3. shows a **Windows notification**.

As soon as you move the mouse or press a key, the alarm stops and the timer starts again.

It **never records what you type or where your mouse is.** It only asks Windows one question: "*how long ago was the last input?*"

02 The big picture



How the four Python files work together

> **Golden rule of the design:** each file has **one job**. The UI doesn't know *how* idle time is measured, and the monitor doesn't know a window exists. This is called **separation of concerns**, and it makes the code easier to test and change.

03 Technologies used, and why

Technology	What it is	Why we chose it
Python 3	Programming language	Simple, readable, has everything needed built in.
Tkinter (`tkinter`, `ttk`)	Python's built-in GUI (window) toolkit	Comes with Python (nothing extra to install), very light, good enough for a small window. Heavier options (Electron, Qt) would use far more memory.

Technology	What it is	Why we chose it
Windows API `GetLastInputInfo`	A function inside Windows (`user32.dll`)	Tells us the time of the last keyboard/mouse input **system-wide**, without reading the input itself. Private and very cheap.
ctypes	Python's built-in way to call C/Windows functions	Lets Python call `GetLastInputInfo` directly, with no extra library.
winsound	Python's built-in Windows sound module	Plays a `.wav` file **asynchronously** (in the background), so the window never freezes.
pystray + **Pillow**	Small libraries for the system-tray icon	Tkinter can't make tray icons itself. pystray does it; Pillow loads the icon image. **Optional**: if missing, the app still works and minimizes to the taskbar.
PyInstaller	Turns a Python program into a `.exe`	So people can run the app **without installing Python**.
Inno Setup	Free tool that builds Windows installers	Creates a normal `Setup.exe` with Start Menu shortcuts, a startup option and an uninstaller.
unittest	Python's built-in test framework	Automatically checks the logic is correct.
JSON	Simple text format for data	Stores the user's settings in a readable file.

Why NOT some other approaches?

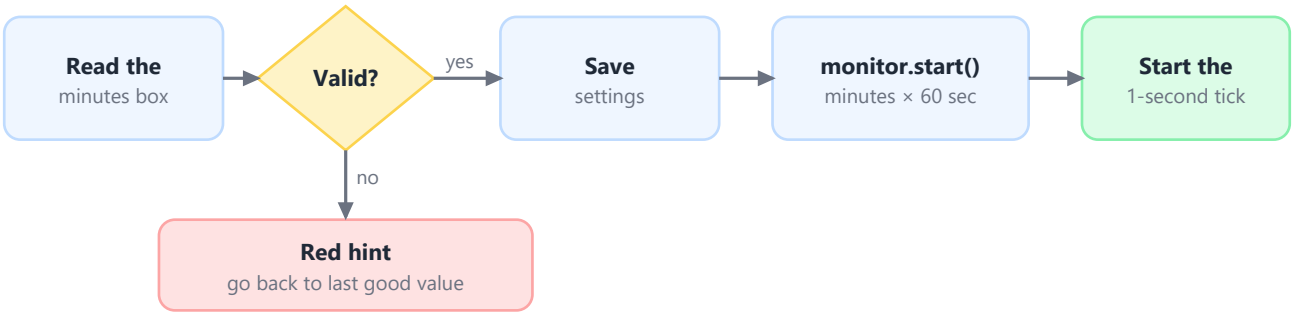
- **Keyboard/mouse hooks** (e.g. the `keyboard` or `pynput` libraries) would *see every key you press*. That behaves like a keylogger, raises privacy concerns, and antivirus tools often flag it. `GetLastInputInfo` only gives a timestamp, which is all we need.
- **A web app or Electron**: much bigger and heavier, and a browser can't see system-wide input.
- **A background service**: overkill. A normal app in the tray is simpler and easy to quit.

04 Files and what each one does

File	Job
`main.py`	The **user interface**: window, buttons, colours, tray icon, alarm reactions, startup flags, single-instance check.
`inactivity_monitor.py`	**Measures idle time** using Windows, and decides **when to fire the alarm** (the "state machine"). Has no UI code.
`alarm.py`	**Plays and stops the sound** without freezing the app. Also finds bundled files (`asset_path`).
`config.py`	**Constants** (presets, limits, version) and **saving/loading settings**. Also parses typed durations like `"7 min"`.
`make_assets.py`	One-time script that **generates** `alarm.wav` (beeps) and `icon.ico` (the blue clock icon).
`assets/`	The sound and icon files that get bundled into the app.
`tests/test_monitor.py`	**Automatic tests** for the logic.
`build.py` / `build.bat`	**Release pipeline**: builds the portable `.exe` and the installer.
`installer.iss`	The **Inno Setup recipe** describing what the installer does.
`requirements.txt`	The list of extra Python packages needed.
`README.md`	How to install, use and build.

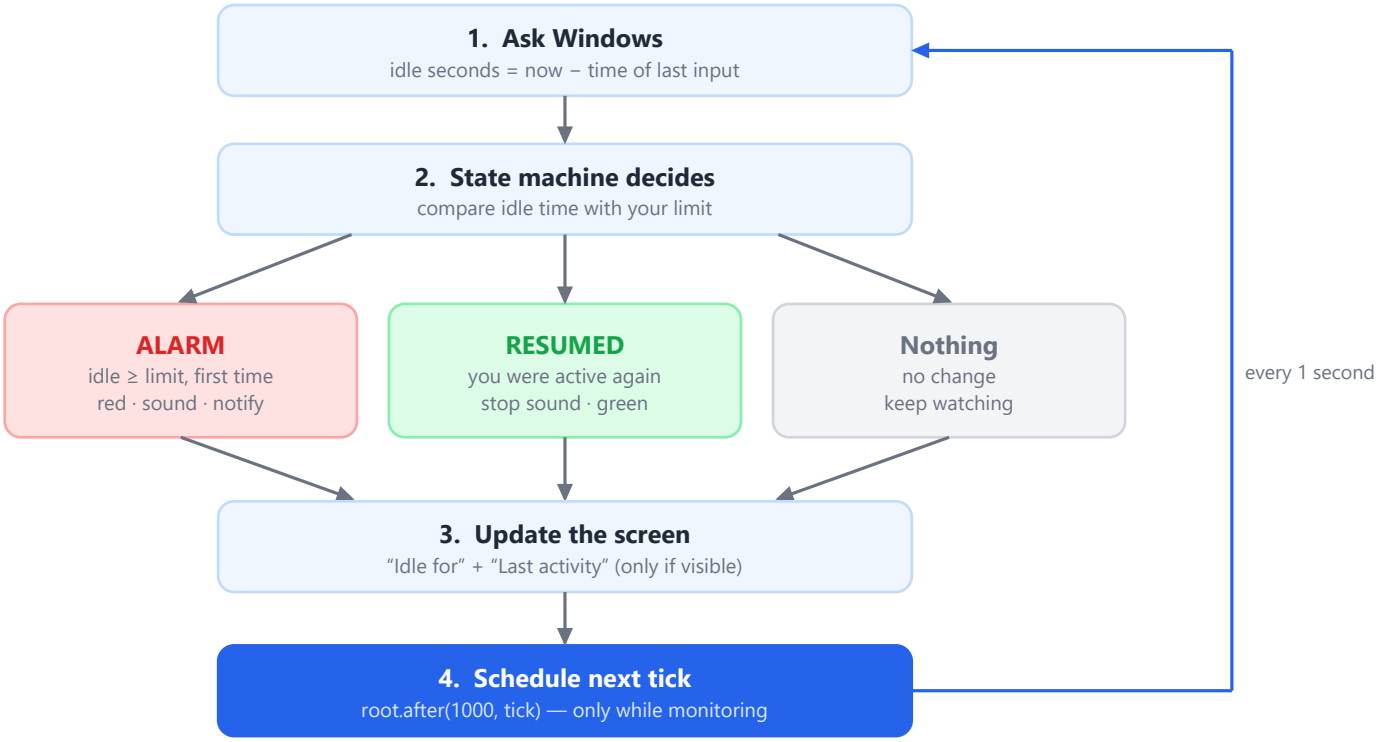
05 How it works, step by step (the main pipeline)

5.1 When you click Start Monitoring



What happens when you click Start Monitoring

5.2 Every second (the "tick")



The 1-second tick: the heart of the app

5.3 When you move the mouse or press a key

Windows updates its "last input" time. On the next tick, idle time is small again, so the monitor sends **RESUMED** and the alarm stops. **You don't have to click anything.**

06 Every feature in simple words

Idle detection

Windows keeps a counter called the **tick count**: milliseconds since the PC started. It also remembers the tick count of the **last input**. Idle time is simply:

```
idle = current tick − last-input tick
```

Duration setting (presets + custom)

You can pick 1, 2, 5, 10 or 15 minutes, or **type your own** (0.5 to 999) and press Enter. `config.parse_minutes()` accepts friendly input like `7`, `7 min`, `1.5` and `2,5`. Bad input shows a red hint and reverts. You can change it **while monitoring** and it applies instantly (`monitor.set_threshold`).

Two alarm modes

- **Alert once**: the sound plays one time.
- **Keep alarming until I'm active**: the sound loops until you move or click **Dismiss Alarm**.

"Fires only once" (the latch)

Without care, the alarm would re-fire every second while you're away. The monitor keeps a flag called `_fired` (a **latch**):

- the alarm fires → `_fired = True` → no more alarms
- the user becomes active → `_fired = False` → ready for the next time

Dismiss

Stops the sound and clears the red state. Because the latch stays set, it won't re-fire until you've actually been active.

Sound on/off + Test sound

Untick **Alarm sound enabled** for a silent (visual-only) alarm. ▶ **Test sound** plays the alarm once so you can hear it. It never interrupts a real alarm.

Runs in the background (system tray)

The app is a **background app**. Pressing **close (X)** or **minimize** only hides the window into the tray (near the clock), and monitoring continues. The first time you close it, a notification explains that it's still running. Double-click the tray icon to bring the window back. **Right-click** the tray icon for a menu: **Open**, **Start/Stop monitoring**, and **Quit**, which is the only way to fully exit. If the tray icon can't be created, X quits normally so the app never becomes invisible.

Notification

When the alarm fires, a Windows notification appears from the tray icon.

Window pops to the front

On alarm the window is restored and made **"topmost"** for 2 seconds so you notice it, then returns to normal.

Only one copy can run

If you open the app again (from the Start Menu, say) while it's already running in the background, the **existing window pops up** and the new copy exits. This uses a **named mutex** and a **named event** (explained below).

Settings are remembered

Your minutes, mode and sound choice are saved to `%APPDATA%\InactivityAlarm\settings.json` and loaded next time.

Startup options

- `--minimized` starts hidden in the tray.
- `--start` starts monitoring immediately.

The installer's **"Start automatically when I sign in"** option uses both.

Sharp on high-resolution screens

The app declares itself **DPI-aware**, so text isn't blurry on 4K or laptop screens, and spacing scales to match.

07 Key concepts you should know

7.1 Windows API and `ctypes`

Windows provides thousands of functions inside DLL files (like `user32.dll`). `ctypes` lets Python call them. We must tell ctypes the **argument types** (`argtypes`) and **return type** (`restype`) so data is passed correctly.

```
_user32 = ctypes.WinDLL("user32", use_last_error=True)
_user32.GetLastInputInfo.argtypes = [ctypes.POINTER(_LASTINPUTINFO)]
_user32.GetLastInputInfo.restype = wintypes.BOOL
```

A **struct** (`_LASTINPUTINFO`) is a small block of memory with named fields that Windows fills in for us.

7.2 The 32-bit tick counter and wraparound

The tick count is a 32-bit number, so its maximum is about 4.29 billion ms ≈ **49.7 days**. After that it **wraps back to 0**, like a car odometer rolling over.

If the last input was at tick 4,294,967,000 and now is tick 500, naive subtraction gives a negative number. We fix it with a **bit mask**:

```
diff = (now - last) & 0xFFFFFFFF   # keeps the result in 32-bit range
```

7.3 The race-condition clamp

We call two functions one after another: first get the last-input time, then get the current time. If you press a key **between** those two calls, "last input" can be a few ms *newer* than "now". The masked difference then becomes ~49 days, which would trigger a false alarm. So any difference over half the range is treated as **0**. A **race condition** is a bug that depends on the exact timing of two things.

7.4 Polling vs events

- **Polling** = asking repeatedly ("idle yet?") on a timer. We poll **once per second**, which is cheap (~0.2–0.4 % of one CPU core).
- **Events/hooks** = being told about every key press. That's more invasive, and we don't need it.

We poll **only while monitoring** to save CPU.

7.5 The event loop (`mainloop`) and `after()`

A GUI program runs an **event loop**: it waits for clicks and redraws, forever. If your code runs for a long time inside that loop, the window **freezes**. So we never use `sleep()` in the UI. Instead, `root.after(1000, self._tick)` says "*run* `_tick` *again in 1 second*", and the loop stays free.

7.6 Threads and thread safety

A **thread** is a separate line of execution running at the same time. pystray runs the tray icon on **its own thread**. **Tkinter is not thread-safe**, so only the main thread may touch the window. Our solution:

```
tray thread:   commands.put("toggle")      ← just drop a message
main thread:   every 200 ms, check the queue and act on messages
```

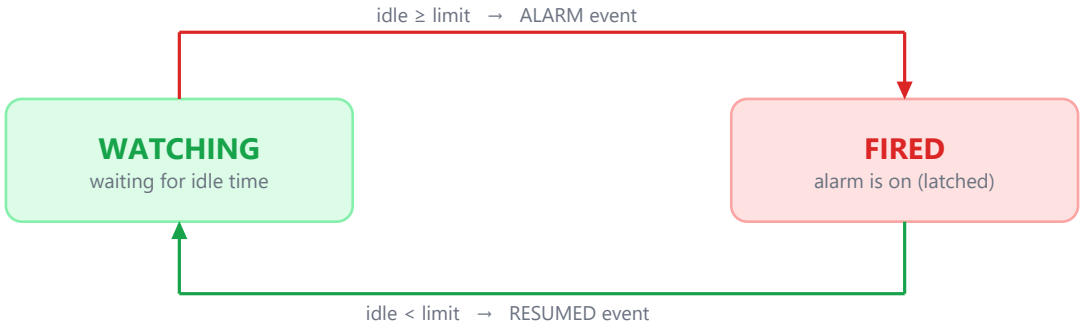
A **queue** (`queue.Queue`) is a thread-safe mailbox. This "producer/consumer" pattern is a standard way to talk between threads safely.

7.7 Asynchronous (non-blocking) sound

`winsound.PlaySound(..., SND_ASYNC)` starts playing and **returns immediately**, so the UI keeps running. `SND_LOOP` repeats the sound. `PlaySound(None, SND_PURGE)` stops it. A `playing` flag prevents **overlapping** alarms.

7.8 State machine

A **state machine** is logic that's always in one of a few states and changes state on specific inputs. Ours, in effect:



The alarm state machine with its “fires once” latch

This makes behaviour predictable and easy to test.

7.9 Named mutex and named event (single instance)

A **mutex** is a lock provided by Windows. We create one with a fixed name (`Local\InactivityAlarm.SingleInstance`). If it **already exists**, Windows returns error 183, which means another copy is running. `Local\` means "per logged-in user session".

A **named event** is a Windows on/off signal that two programs can share by name. The running copy creates `Local\InactivityAlarm.Show` and checks it every 200 ms. A second launch **sets** the event and exits, and the running copy sees it and shows its window. This is how "open it again" brings back the background app.

```
second launch:  mutex exists?  ──yes──►  SetEvent("Show")  ──►  exit
running copy:   every 200 ms:  event set?  ──yes──►  show window
```

7.10 Atomic file write

Settings are written to `settings.json.tmp` first, then renamed over the real file with `os.replace`. If the PC crashes mid-write, the old file is still intact and never half-written. This is called an **atomic** write.

7.11 Validation and safe defaults

Every setting loaded from disk is checked (right type? in range?). Anything wrong falls back to the default, so a corrupted file can never crash the app.

7.12 Dependency injection (for testing)

`InactivityMonitor(idle_source=...)` takes the idle-measuring function as a parameter. The real app passes the Windows function, while tests pass a **fake** that returns any number we want. That lets us test "what happens at 59, 60, 61 seconds idle" instantly, without waiting.

7.13 Graceful degradation

If something optional fails, the app keeps working with less:

- no pystray → no tray icon, but the app still works
- no audio device → the visual alarm still works
- Windows API error → a message is shown, and the alarm never fires by mistake

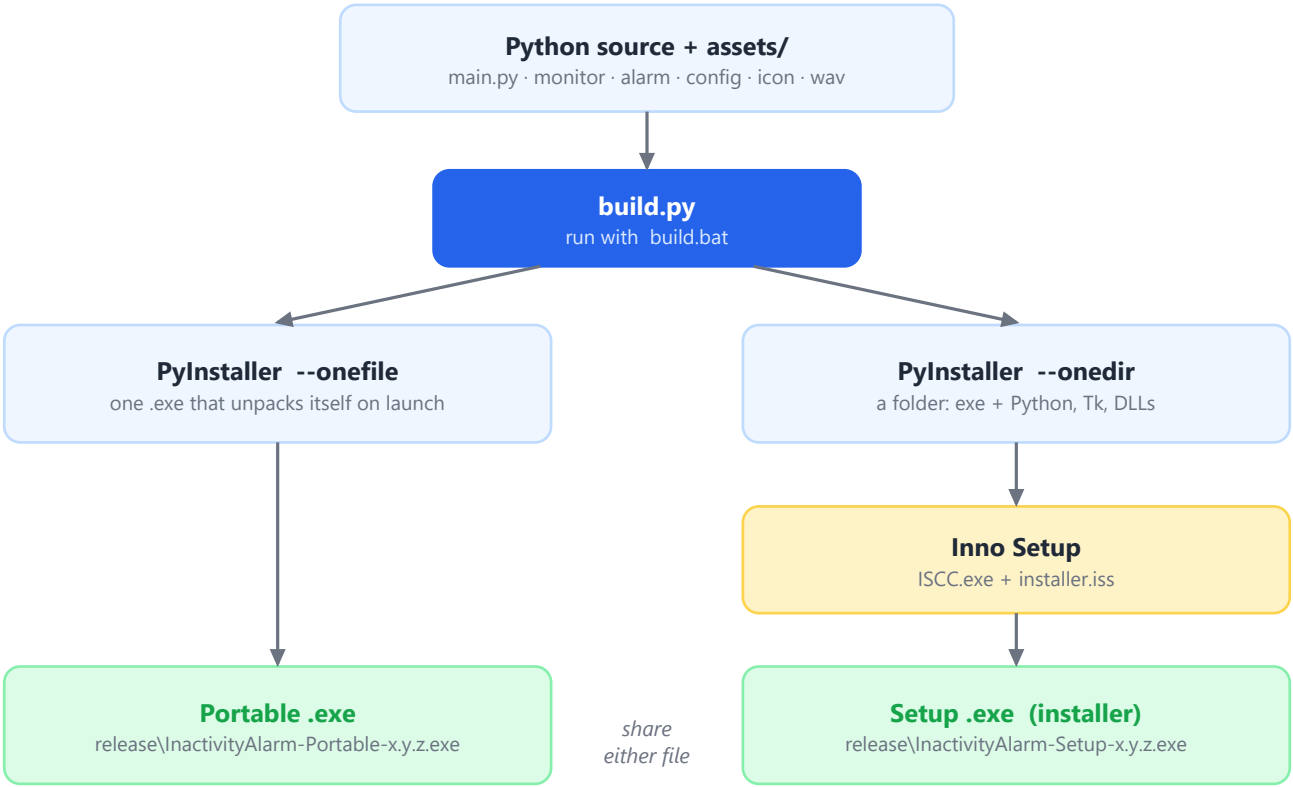
7.14 DPI awareness

DPI = dots per inch. On high-resolution screens Windows may stretch "unaware" apps, making them blurry. `SetProcessDpiAwareness(1)` tells Windows we handle scaling ourselves, and `px()` scales our spacing.

7.15 Bundled files and `sys._MEIPASS`

Inside a PyInstaller `.exe`, our `assets/` folder is unpacked to a temporary folder whose path is in `sys._MEIPASS`. `asset_path()` checks for it, so the same code finds `alarm.wav` both when run as `python main.py` and as an `.exe`.

08 The build and release pipeline



From source code to shareable release files

Key build terms

- **PyInstaller** packs Python itself, our code, Tkinter and every needed DLL (including `python313.dll` and the Visual C++ runtime `VCRUNTIME140.dll`) into the output, so **the user needs nothing installed**.
- `--onefile` **vs** `--onedir`:
  - *onefile* = one tidy `.exe`, but it unpacks itself on every launch. That's a bit slower to start, and antivirus tools are more suspicious of it.
  - *onedir* = a folder, which starts faster and is better inside an installer.
- `--windowed`: don't show a black console window.
- `--add-data "assets;assets"`: bundle the sound and icon.
- **Version info** (`--version-file`): stamps the name, version and publisher into the `.exe` (you can see it under right-click → Properties → Details).

What the installer (`installer.iss`) does

Setting	Meaning
`PrivilegesRequired=lowest`	Installs **per user**, so **no admin rights** are needed. Goes to `%LOCALAPPDATA%\Programs\Inactivity Alarm`.
`[Tasks]`	Checkboxes in the wizard: Desktop icon, Start at sign-in (both ticked by default).
`[Icons]`	Creates the Start Menu and Desktop shortcuts.
`[Registry]` → `...\CurrentVersion\Run`	The standard Windows "start at sign-in" list.
`PrepareToInstall` (code)	Closes a running copy first, so upgrades work.
`[UninstallRun]` / `[UninstallDelete]`	On uninstall: stop the app, remove the settings folder.
`AppId` (a GUID)	A unique ID, so new versions **upgrade** the old one instead of installing twice.
`MinVersion=10.0`, `x64compatible`	Requires Windows 10/11, 64-bit (Python 3.13 needs this).

Making a new version

1. Change `APP_VERSION` in `config.py` (e.g. `"1.1.0"`).
2. Run `build.bat`.
3. Share the new files in `release\`.

09 The testing pipeline

Level	What it checks	How
Unit tests (`tests/test_monitor.py`)	Wraparound maths, race clamp, alarm fires once, re-arms after activity, threshold changes, error handling, parsing `"7 min"`, settings load/save and corrupt files	`python -m unittest discover tests -v`
Real Windows API check	Idle time really grows, and a real (simulated) key press and mouse move really reset it	Scripted test using `keybd_event` / `mouse_event`
GUI test harness	Real window with fake idle time: buttons, colours, alarm/dismiss, tray hide/show, clean quit with no leftover threads	Script driving the app

Level	What it checks	How
Exe and installer tests	Exe runs without Python, single instance works, install/upgrade/uninstall leave nothing behind	Silent install (`/VERYSILENT`) plus checks

> **Why test at different levels?** Unit tests are fast and precise but use fakes. The real-API and exe tests prove the pieces work together on actual Windows.

10 Sharing the app: things to know

- **No Python needed** on other PCs; everything is bundled.
- **Requirements:** Windows 10 or 11, 64-bit.
- **SmartScreen warning ("Windows protected your PC")** appears because the app isn't **code-signed**. Users click **More info** → **Run anyway**.
- **Code signing** = a digital signature proving who made the file. It needs a paid certificate (or Azure Trusted Signing). It removes most warnings and builds trust.
- **Antivirus false positives:** unsigned PyInstaller apps are sometimes flagged by mistake. The installer (onedir) version is less likely to be flagged than the portable one.

11 Privacy by design

Not done	Why it matters
No keystroke reading	Can't leak passwords or messages
No mouse-position tracking	No behaviour profiling
No screenshots	Nothing visual captured
No history / logs	Nothing stored about your activity
No internet	Nothing ever leaves your PC

The **only** file written is the small settings file.

12 Known limitations

- **Software-generated input counts as activity**: mouse-jiggler tools, remote-desktop input and some controller apps can stop the alarm.
- **Lock screen / UAC prompts**: input there isn't visible to the app.
- **Per session**: only input in your own Windows login is seen.
- **Do Not Disturb / Focus Assist** can hide the notification (sound and window still work).
- **Windows 11** may hide new tray icons under the `^` arrow.

13 Glossary (quick reference)

Keyword	Simple meaning
API	A set of functions one program offers for others to use.
Asynchronous / non-blocking	Starts a task and continues immediately without waiting.
Atomic write	Save that either fully happens or doesn't happen at all.
Bit mask (`& 0xFFFFFFFF`)	Keeps only the lowest 32 bits of a number.
Bootloader	The small starter part of a onefile exe that unpacks and launches the app.
Code signing	Digitally signing an exe to prove the publisher.
ctypes	Python module for calling C/Windows functions.
Dependency injection	Passing in a part (like the idle function) instead of hard-coding it, which makes testing easy.
DLL	A Windows library file containing functions (e.g. `user32.dll`).
DPI awareness	Telling Windows the app handles high-resolution scaling itself.
Event loop / `mainloop`	The GUI's endless loop that handles clicks and redraws.
GUID	A globally unique ID (used as the installer's `AppId`).
Graceful degradation	Keep working with fewer features when something fails.
Idle time	Time since the last keyboard/mouse input.
Inno Setup	Tool that builds Windows installers from a `.iss` script.
JSON	Simple text format for storing data.
Latch	A flag that stays set until a specific reset condition.
Fingerprint / watermark	A unique hidden marker that proves where a copy came from.
License	The legal terms that say what others may and may not do with your work.
Mutex	A Windows lock; a named one lets us detect another running copy.
Named event	A Windows on/off signal shared by name between programs (used to say "show your window").
Background app	An app that keeps running with its window hidden, usually from the system tray.
onefile / onedir	PyInstaller output as one exe vs. a folder.
Per-user install	Installs only for the current user; no admin needed.
Polling	Checking something repeatedly on a timer.
PyInstaller	Packs a Python app plus Python itself into an exe.
Queue	Thread-safe mailbox for passing messages between threads.
Race condition	A bug caused by the exact timing of two operations.
Registry `Run` key	Windows list of programs to start at sign-in.
Separation of concerns	Each part of the code has one clear job.
SmartScreen	Windows feature that warns about unknown or unsigned downloads.

Keyword	Simple meaning
State machine	Logic that moves between defined states on defined events.
Struct	A block of memory with named fields (used with Windows APIs).
System tray	Icon area near the clock in the taskbar.
Thread / thread safety	Parallel execution / code that's safe when used from several threads.
Tick count	Milliseconds since Windows started (32-bit, wraps every ~49.7 days).
Tkinter / ttk	Python's built-in GUI toolkit / its themed widgets.
Topmost	A window setting that keeps it above all others.
Unit test	Small automatic test of one piece of logic.
Wraparound	A counter overflowing back to 0 (like an odometer).
`sys._MEIPASS`	Folder where a PyInstaller exe unpacks its bundled files.
`%APPDATA%` / `%LOCALAPPDATA%`	Per-user folders for app settings and per-user programs.

14 Common changes: where to edit

I want to…	Edit
Change the version	`APP_VERSION` in `config.py`
Change the publisher name	`PUBLISHER` in `config.py`
Change preset minutes or the default	`PRESET_MINUTES`, `DEFAULT_MINUTES` in `config.py`
Change the alarm sound	Replace `assets/alarm.wav` (or edit `make_wav` in `make_assets.py` and run it)
Change the icon	Replace `assets/icon.ico` (or edit `make_icon`)
Change colours or fonts	Constants at the top of `main.py` (`BG`, `CARD`, `FONT`)
Change the polling speed	`TICK_MS` in `main.py`

15 Ownership, license and fingerprint

The app is **made by Ishant, © 2026 IMPS. All rights reserved.** (see `LICENSE`).

- **Visible credit:** the window footer, the installer, Settings → Apps (publisher **IMPS**) and the exe's Properties → Details.
- **License:** "all rights reserved". People may view and learn from the code and use official releases, but copying or redistributing it needs permission. The installer shows the license before installing.
- **Fingerprint** `IMPS-IA-E91E8AB6F47B4248`: a unique ID hidden in every source file header, the exe's *Comments* field, the alarm sound's metadata and these docs. If you find a suspicious copy, look for it. Search GitHub code for it to find copied repos.

Concept: watermarking vs DRM. A watermark doesn't stop copying. It proves *where something came from*. Truly preventing copying (DRM, obfuscation) is complex and can still be broken. For a learning project, clear credit + a license + a fingerprint is the practical balance.

16 Command cheat sheet

```
# run from source
python main.py
python main.py --minimized --start

# run tests
python -m unittest discover tests -v

# regenerate sound + icon
python make_assets.py

# build release files (portable exe + installer) into release\
.\build.bat
```